

ネットワークの学習方法

Tatsuro Senga

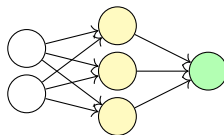
Keio

前回の続き

知っていること：

- ニューラルネットワーク = ニューロンの層
- 単一パーセプトロン：線形問題に限定
- 多層ネットワーク：XOR を解決
- 隠れ層：新しい表現を作成

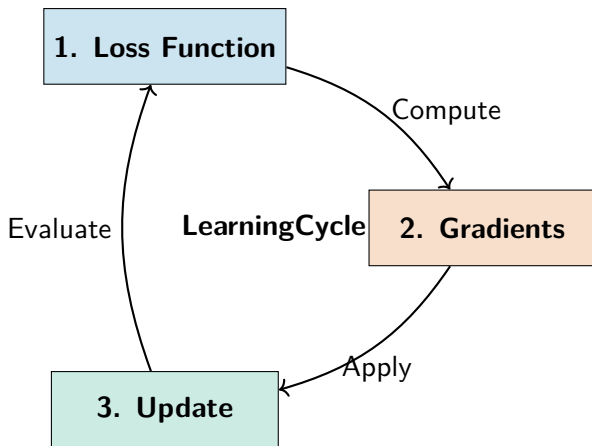
しかし重みをどう見つける？



メイントピック

ネットワークがこれらの接続の正しい重みとバイアスを自動的に見つけるにはどうすればよいか？

学習の三つの柱



損失を定義：
どれだけ間違っているか？

勾配を計算：
どの方向に改善するか？

パラメータを更新：
一歩進む

「どれだけ間違っているか」を定量化

平均二乗誤差 (MSE) :

$$E = \frac{1}{2} \sum_j (y_j^{(L)} - d_j)^2$$

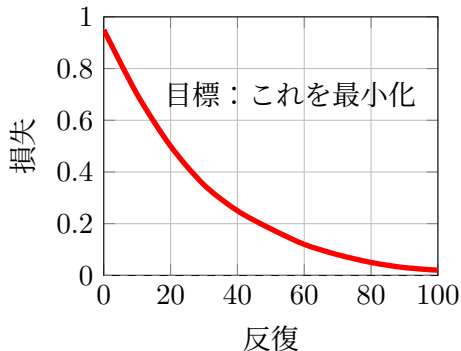
where:

- $y_j^{(L)}$: ネットワークの出力
- d_j : 望ましい出力 (真値)
- E : 最小化すべき総誤差

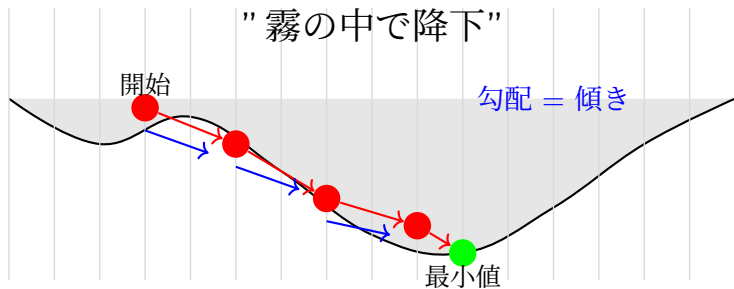
その他の損失関数 :

- クロスエントロピー (分類)
- Huber 損失 (ロバスト回帰)
- カスタム物理情報損失

典型的な損失曲線



谷を見つける



戦略：各位置で傾き（勾配）を測定し、最も急な下向きの方角に一歩進む

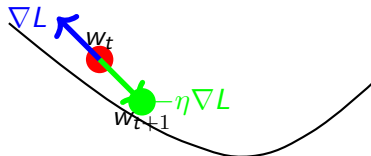
勾配降下法

更新ルール

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

構成要素：

- w_t : 現在の重み
- $\nabla L(w_t)$: 損失の勾配（傾き）
- η : 学習率（ステップサイズ）
- w_{t+1} : 更新された重み

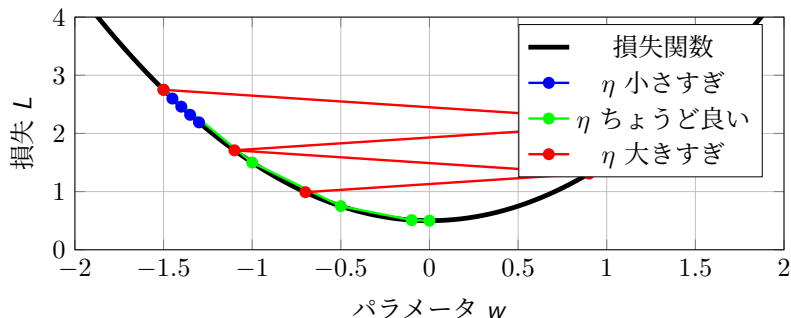


なぜマイナス記号？

- 勾配は上向きを指す
- 下向きに行きたい
- だから勾配と反対方向に進む

損失曲面

学習率



小さすぎ：

収束が遅い
多くの反復が必要

ちょうど良い：

着実な進歩
最小値に到達

大きすぎ：

オーバーシュート
発散する可能性

速度と安定性のバランス

バッチ GD

すべてのデータを使用

- ✓ 正確な勾配
- ✓ 安定
- × 非常に遅い
- × メモリ集約的

確率的 GD

一つのサンプルを使用

- ✓ 非常に高速
- ✓ オンライン学習
- × ノイジー
- × 不安定

ミニバッチ GD

小さなバッチを使用

- ✓ 良いバランス
- ✓ GPU 効率的
- ✓ ノイズが少ない
- ✓ 現代の標準！

典型的なバッチサイズ：32, 64, 128, 256

勾配をどう計算するか？

課題

何百万もの重みが多く層に分散している。
各重みの ∇L を効率的にどう計算するか？

ナイーブなアプローチ：

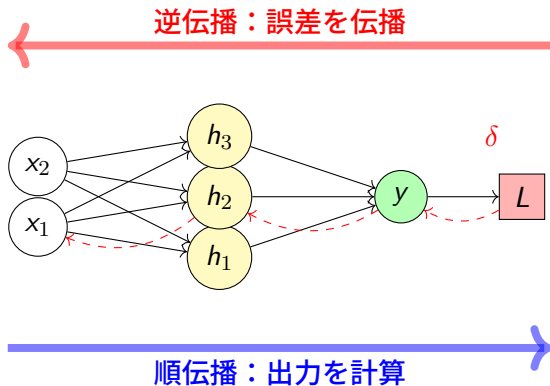
- 各重みを少しずつ摂動
- 損失の変化を測定
- 勾配を推定
- 時間： $O(\text{重み}^2)$

バックプロパゲーション：

- 連鎖律を体系的に使用
- 中間計算を再利用
- 単一の後向きパス
- 時間： $O(\text{重み})$

バックプロパゲーション = 連鎖律による効率的な勾配計算

順伝播 → 逆伝播



1. 順伝播：

- 入力 → 隠れ → 出力
- すべての中間値を保存
- 最終損失を計算

2. 逆伝播：

- 損失 → 出力 → 隠れ → 入力
- 誤差信号 (δ) を計算
- 重み勾配を計算

連鎖律の動作

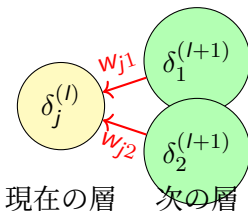
出力層について：

$$\frac{\partial E}{\partial w_{ij}^{(L)}} = \frac{\partial E}{\partial y_j^{(L)}} \cdot \frac{\partial y_j^{(L)}}{\partial w_{ij}^{(L)}} \quad (1)$$

$$= \underbrace{(y_j^{(L)} - d_j)}_{\text{誤差信号 } \delta_j^{(L)}} \cdot \underbrace{y_i^{(L-1)}}_{\text{前の出力}} \quad (2)$$

隠れ層について：

$$\delta_j^{(l)} = \left(\sum_k \delta_k^{(l+1)} w_{jk}^{(l+1)} \right) \cdot f'(z_j^{(l)}) \quad (3)$$



確率的勾配降下法

甘利（1967） - 基盤：

オンライン学習ルール

$$\Delta\theta = -\eta \frac{\partial \ell(x, \theta)}{\partial \theta}$$

重要な洞察：

- ニューラルネットワークの確率的近似
- 収束証明
- 最適学習率
- 情報幾何学的解釈

この研究は有名なバックプロパゲーション論文より約 20 年早い！

酔歩の再考

” 酔歩を行なう状況に似ている。彼は平均としては山をずり落ちて行き、ついには谷底へ落ち込む”

— 酔った人の歩行のように、平均的に山を下って谷に到達

現代への影響：

- SGD がまだ支配的
- Adam, RMSprop の基盤
- ディープラーニングの基盤

重要な要点：

- 勾配降下法：最適化戦略（谷を見つける）
- バックプロパゲーション：勾配計算器（効率的な連鎖律）
- 学習率：重要なハイパーパラメータ（ステップサイズ）
- ミニバッチ：実用的な妥協（速度 vs 安定性）

ODE ソルバーの実装

ネットワークの学習方法を理解したので、微分方程式を解くニューラルネットワークを構築

やること：

- MATLAB で学習アルゴリズムを実装
- ODE の試行解を設計
- ソルバーをデバッグ・最適化

宿題：

- ① 勾配降下法の方程式を復習
- ② 2 層ネットワークの連鎖律を導出してみる
- ③ MATLAB 最適化ツールボックスをインストール